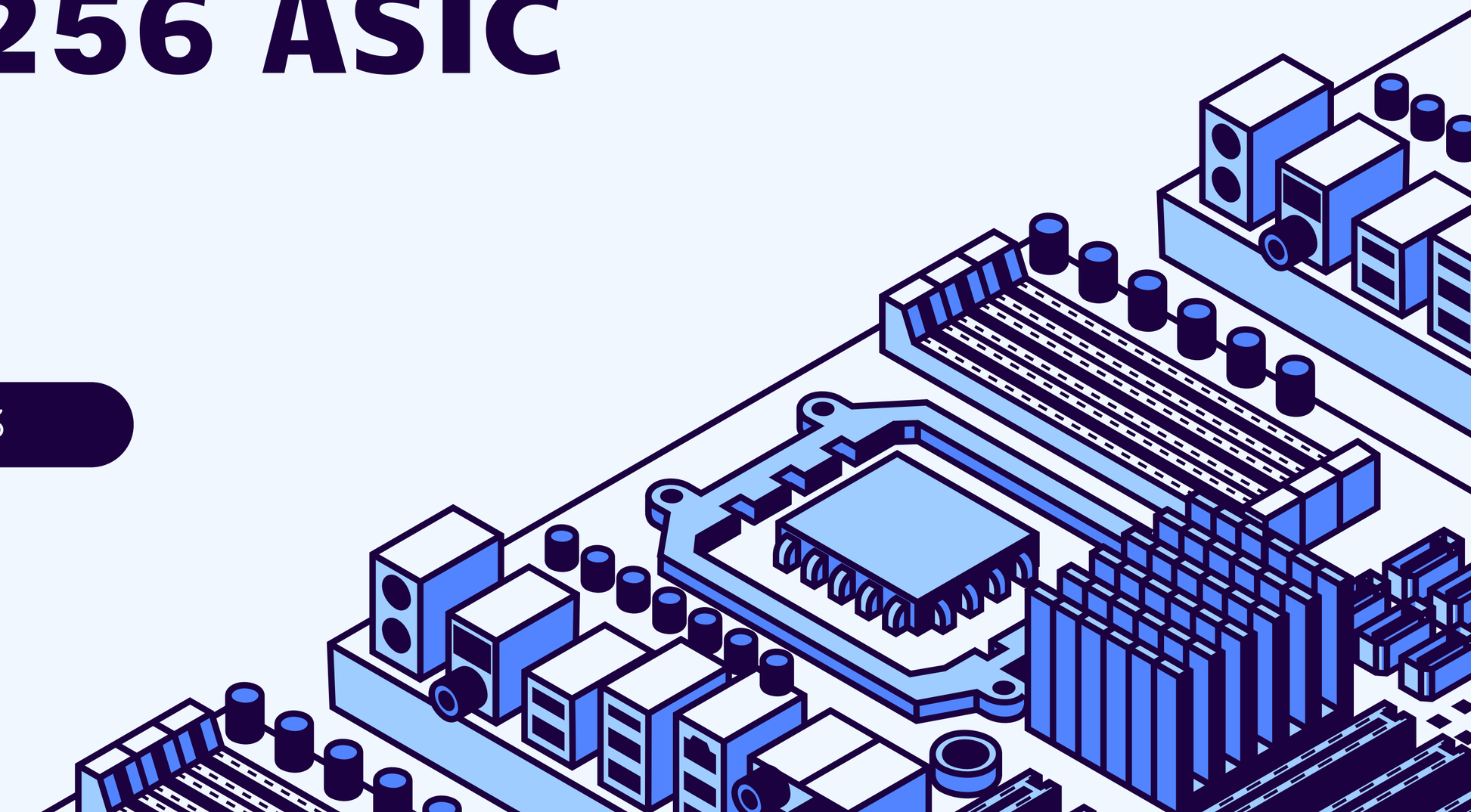
Hash Brown - SHA-256 ASIC

VLSI Team 3

Oskar Bushrod
Beth Cham
Ronit Ravi
Hrishikesh Venkatesh

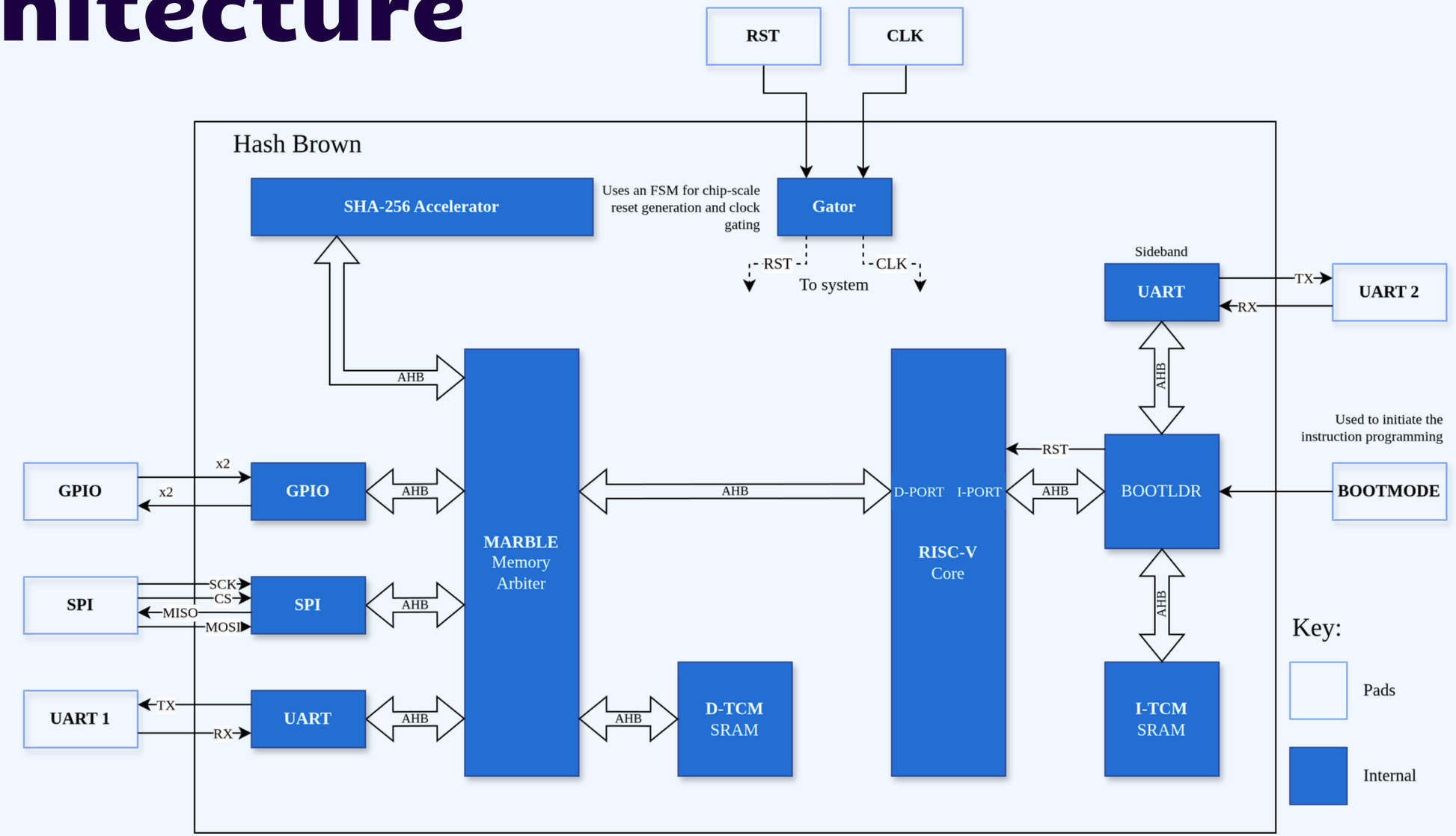


Introduction

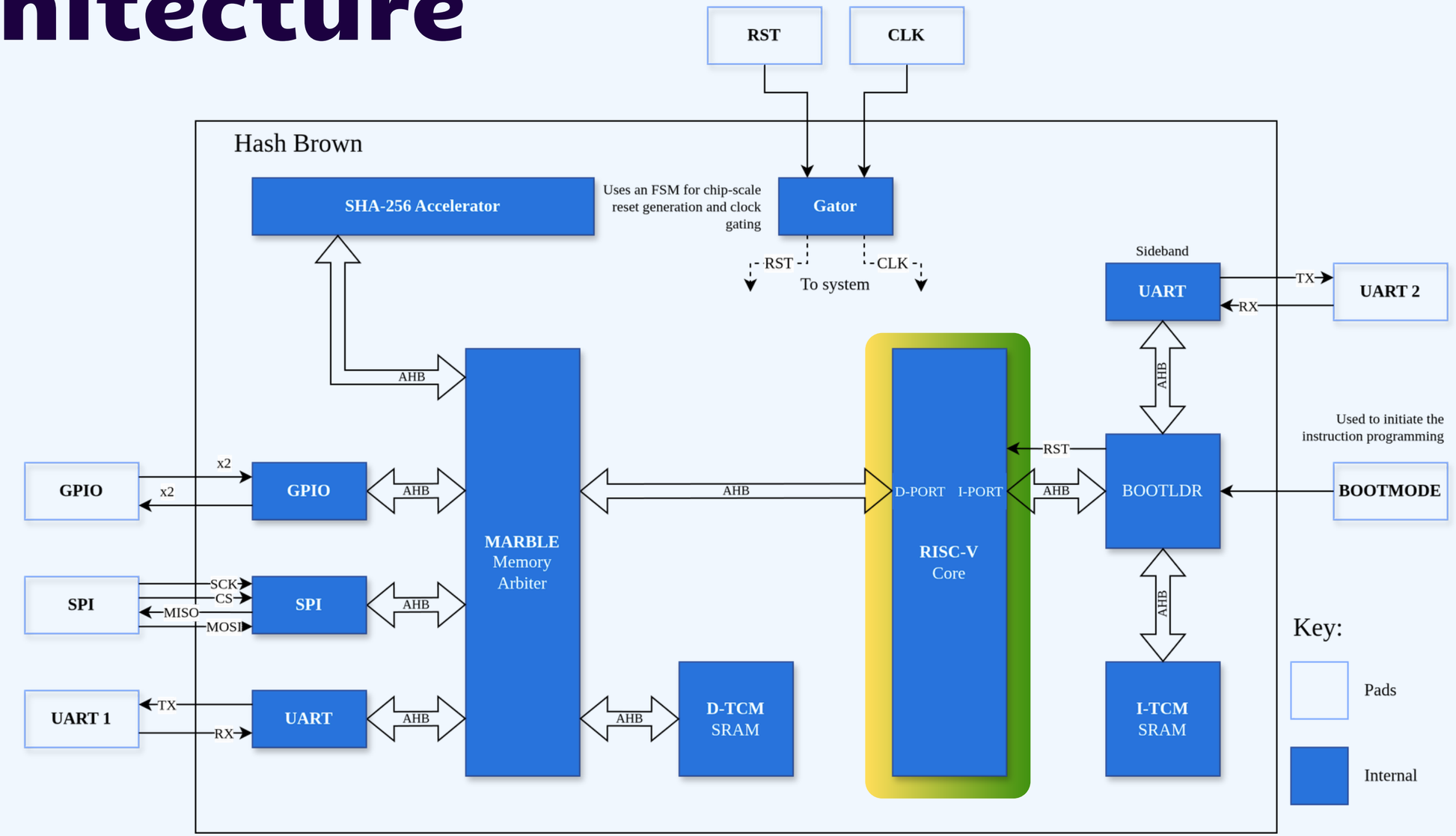
An ASIC that targets the acceleration of the SHA-256 algorithm with cryptographic applications. It has the following features:

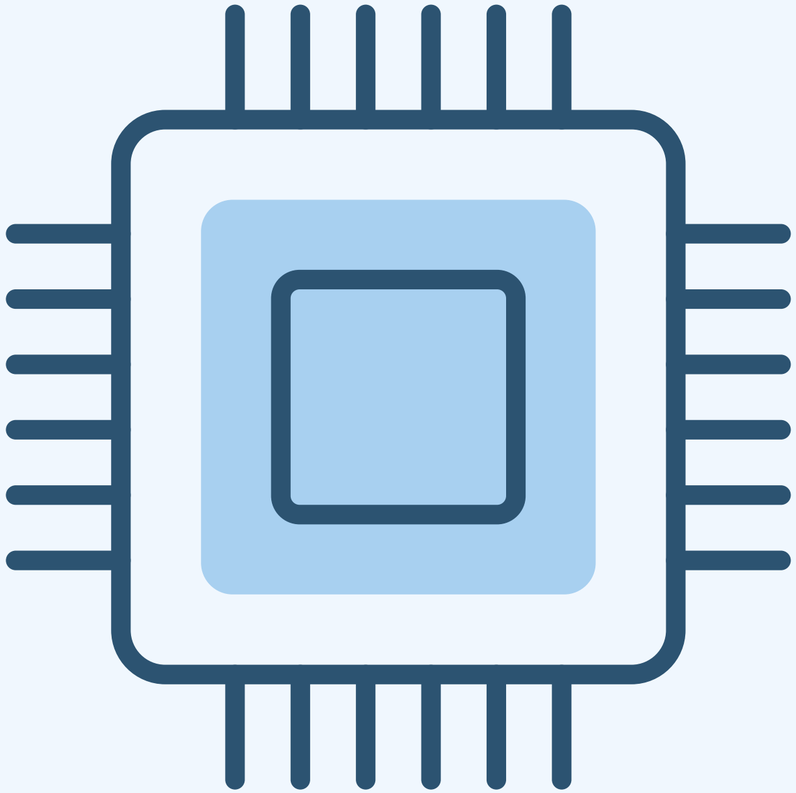
- Verified RV32I Core (w/ Bootloader)
- SHA-256 Hardware Accelerator
- 16 KB Instruction SRAM
- 1 KB Data SRAM
- AHB-Lite Communication
- UART, SPI, GPIO Interfaces
- Operates at 250 MHz

Architecture



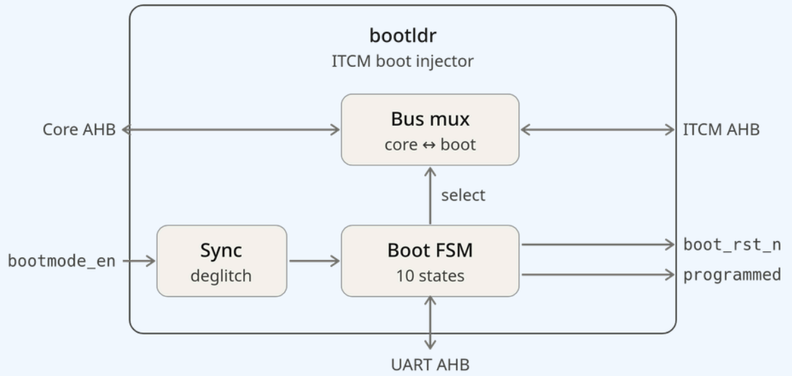
Architecture



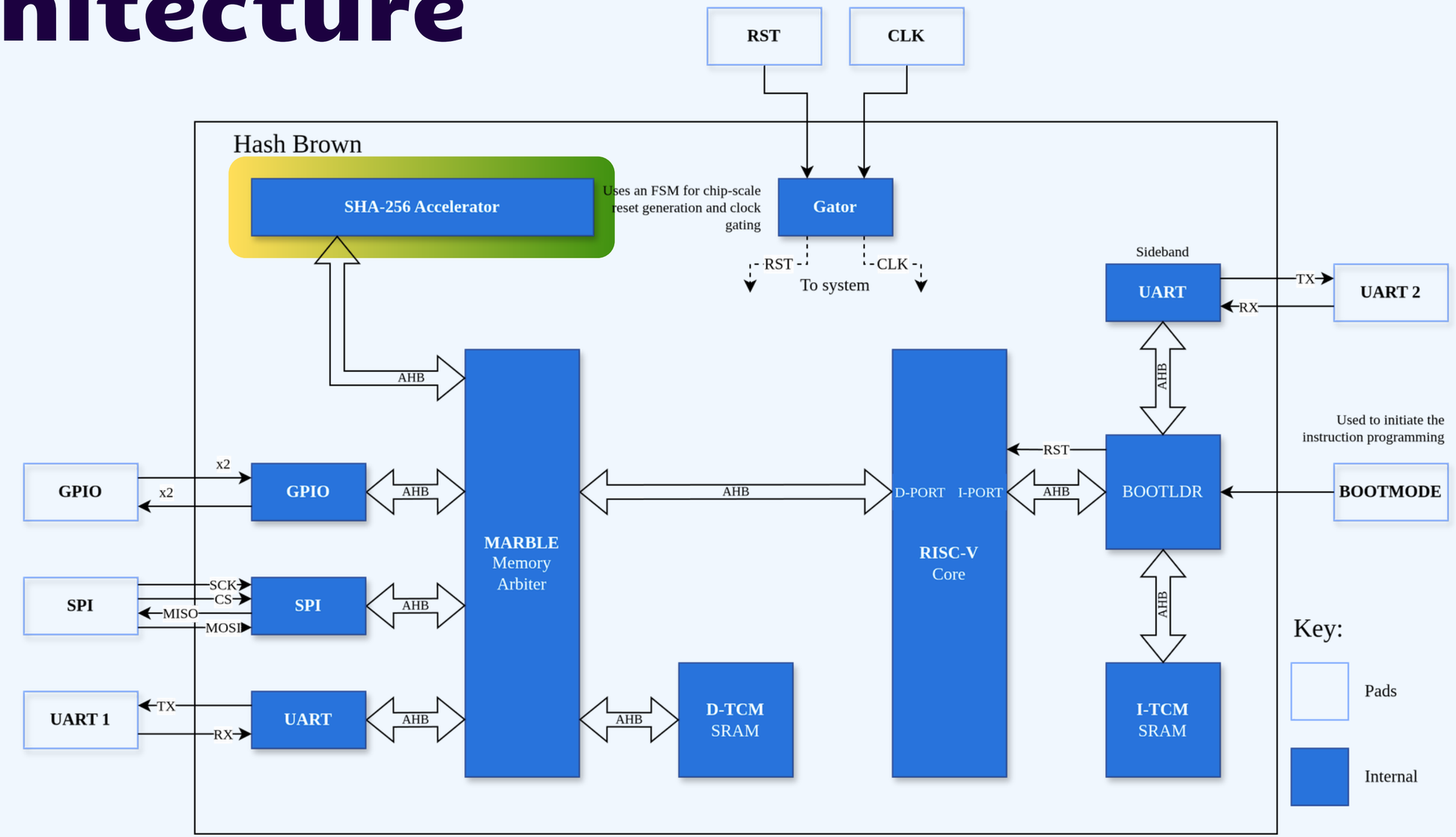


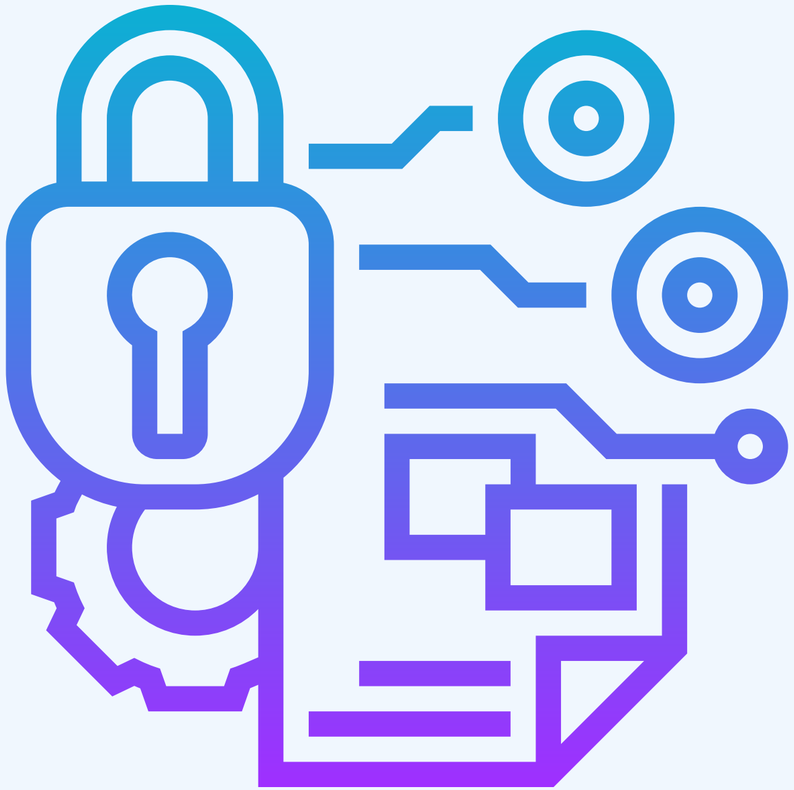
RISC-V32I

- Implements the core RV32I instruction set.
- Adapted from the core developed in the second year IAC course.
- Inherits hazard unit and pipelining from that design.
- Removal of the M extension.
- Extracted data and instruction memory.
- Formally verified.



Architecture





SHA-256 Accelerator

- Cryptographic algorithm.
- Accelerates computation time.
- Verified through cocotb covergroups and golden models.
- Key in modern security due to its 3 main properties
    a. One-way: Easy to compute hash, infeasible to recover message
    b. Determinisitic: Same input always produces same output.
    c. Avalanche: A small change in input produces a large change in output.
- Takes 128 cycles for a 512-bit message at 250Mhz
    - RISCV32 takes 1200-1300 cycles at 250Mhz
    - ~1000 cycles faster

SHA Algorithm

SHA-256 — all words are 32-bit; all "+" is addition mod 2^32
Legend: ⋙ rotate-right ≫ shift-right ⊕ XOR ⋀ AND ¬ NOT

Constants H0..H7 : 8 initial hash values (√ of first 8 primes)
K[0..63]: 64 round constants (∛ of first 64 primes)

Padding [message | 1 | zeros | length L] → total length × of 512 bits

Per 512-bit chunk
copy chunk into w[0..15]
for i = 16..63: # expand schedule
  w[i] = w[i-16] + σ0(w[i-15]) + w[i-7] + σ1(w[i-2])
  a..h = H0..H7
for i = 0..63: # compression
  T1 = h + Σ1(e) + Ch(e,f,g) + K[i] + w[i]
  T2 = Σ0(a) + Maj(a,b,c)
  h=g g=f f=e e=d+T1
  d=c c=b b=a a=T1+T2
  H0..H7 += a..h

Output digest = H0 ‖ H1 ‖ … ‖ H7 (256 bits)

$\sigma0(x)=(x\ggg7)\oplus(x\ggg18)\oplus(x\gg3)$
$\Sigma0(x)=(x\ggg2)\oplus(x\ggg13)\oplus(x\ggg22)$
$\sigma1(x)=(x\ggg17)\oplus(x\ggg19)\oplus(x\gg10)$

$\Sigma1(x)=(x\ggg6)\oplus(x\ggg11)\oplus(x\ggg25)$
$Ch(e,f,g)=(e\wedge f)\oplus(\neg e\wedge g)$
$Maj(a,b,c)=(a\wedge b)\oplus(a\wedge c)\oplus(b\wedge c)$

SHA Algorithm

SHA-256 — all words are 32-bit; all "+" is addition mod 2^32
Legend: ⋙ rotate-right ≫ shift-right ⊕ XOR ∧ AND ¬ NOT

—— Constants ——
H0..H7 : 8 initial hash values (frac. parts of √ of first 8 primes)
K[0..63]: 64 round constants (frac. parts of ∛ of first 64 primes)

H0=6a09e667 H1=bb67ae85 H2=3c6ef372 H3=a54ff53a
H4=510e527f H5=9b05688c H6=1f83d9ab H7=5be0cd19

—— Padding ——
[message (L bits) | 1 | K zeros | L as 64-bit int]
append '1', then K zeros so (L + 1 + K + 64) ≡ 0 (mod 512),
then L as a 64-bit big-endian integer → length is a multiple of 512

—— Per 512-bit chunk ——
copy chunk into w[0..15]

```
# Message schedule
for i = 16..63:
s0 = (w[i-15] ⋙ 7) ⊕ (w[i-15] ⋙ 18) ⊕ (w[i-15] ≫ 3)
s1 = (w[i-2] ⋙ 17) ⊕ (w[i-2] ⋙ 19) ⊕ (w[i-2] ≫ 10)
w[i] = w[i-16] + s0 + w[i-7] + s1

# Init working variables
a..h = H0..H7

# Compression loop
for i = 0..63:
S1 = (e ⋙ 6) ⊕ (e ⋙ 11) ⊕ (e ⋙ 25)
ch = (e ∧ f) ⊕ (¬e ∧ g)
T1 = h + S1 + ch + K[i] + w[i]
S0 = (a ⋙ 2) ⊕ (a ⋙ 13) ⊕ (a ⋙ 22)
maj = (a ∧ b) ⊕ (a ∧ c) ⊕ (b ∧ c)
T2 = S0 + maj

h=g g=f f=e e=d+T1
d=c c=b b=a a=T1+T2

# Fold chunk into hash
H0..H7 += a..h
```

—— Output ——
digest = H0 ‖ H1 ‖ H2 ‖ H3 ‖ H4 ‖ H5 ‖ H6 ‖ H7 (256 bits)

SHA Architecture

Pre-processor

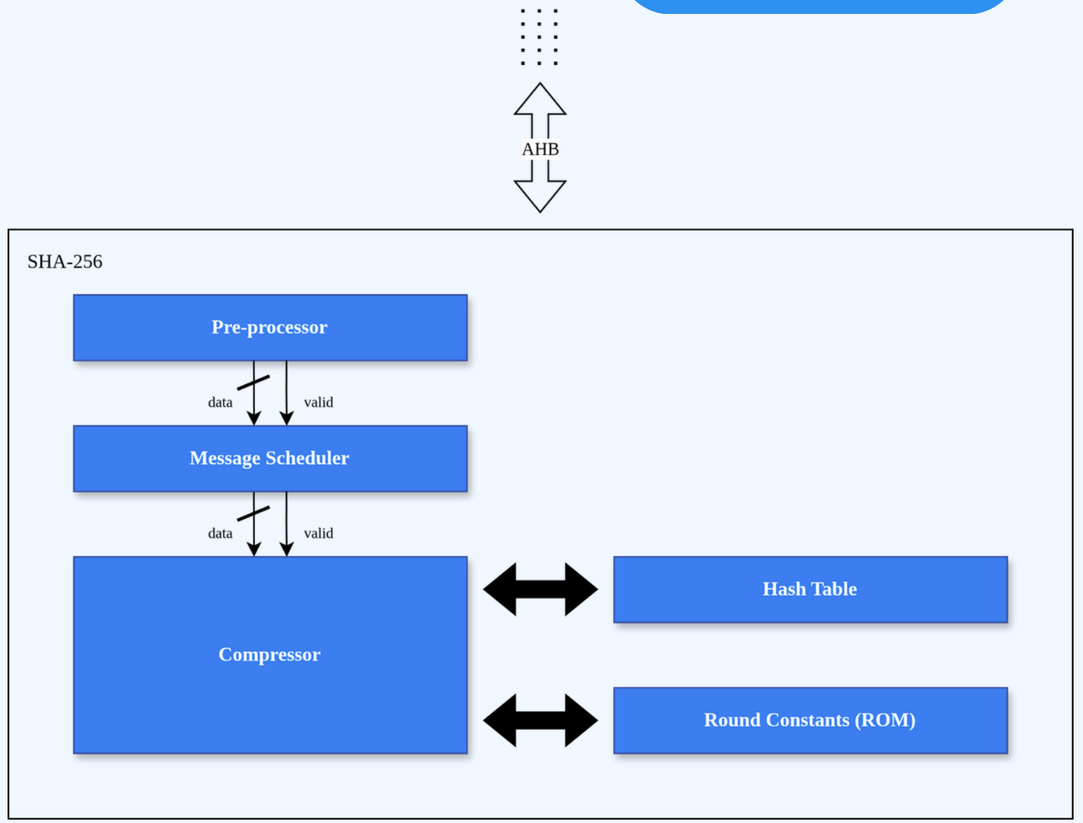
- Streams in 8-bit bytes, accumulates 512-bit blocks
- Implements SHA padding in hardware
- FSM handles accumulation/sending stages

Message scheduler

- Generates W[0..63]
- Optimization: 16-word circular buffer instead of 64 registers, computing new words on the fly
    - Significant area saving

Compressor

- 64 rounds over working variables a..h with initial hash constants
- 2-stage pipelined
    - Accumulates digest across multiple blocks
- Asserts o_done with the final 256-bit hash



SHA Architecture

I/O

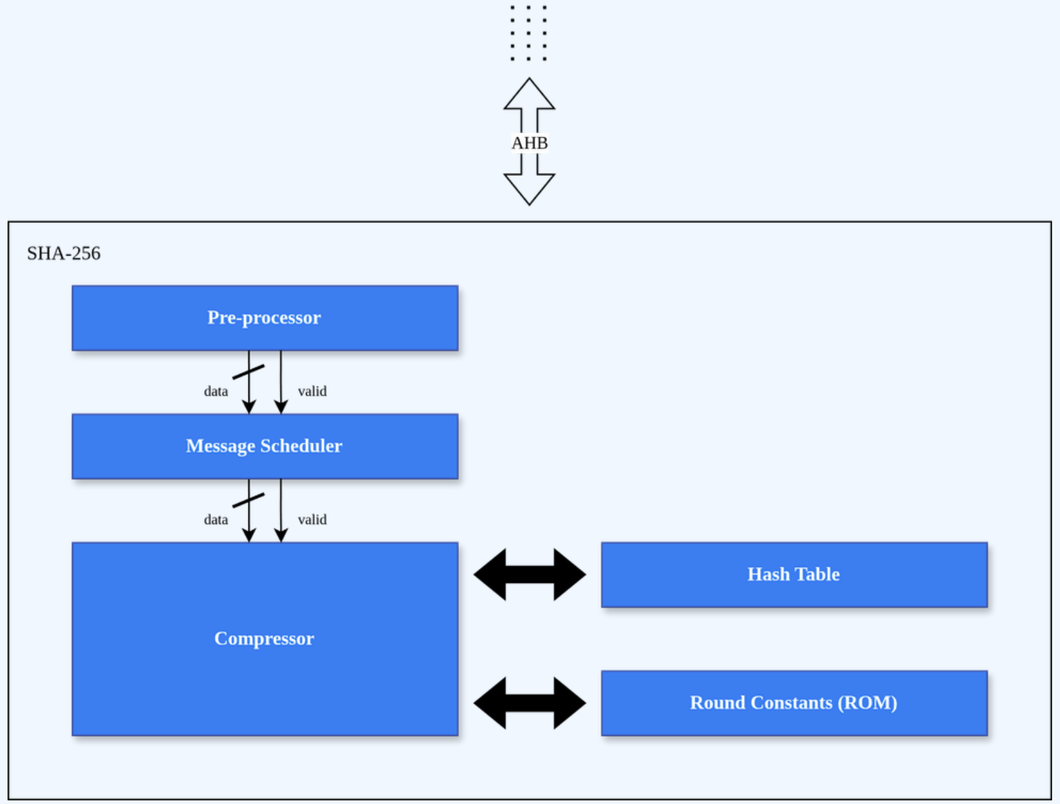
- In: 8-bit i_message + valid / last handshake (byte-serial stream)
- Out: 256-bit o_hash + o_done
- Streaming interface
  - No need to buffer the whole message

AHB-Lite memory-mapped slave

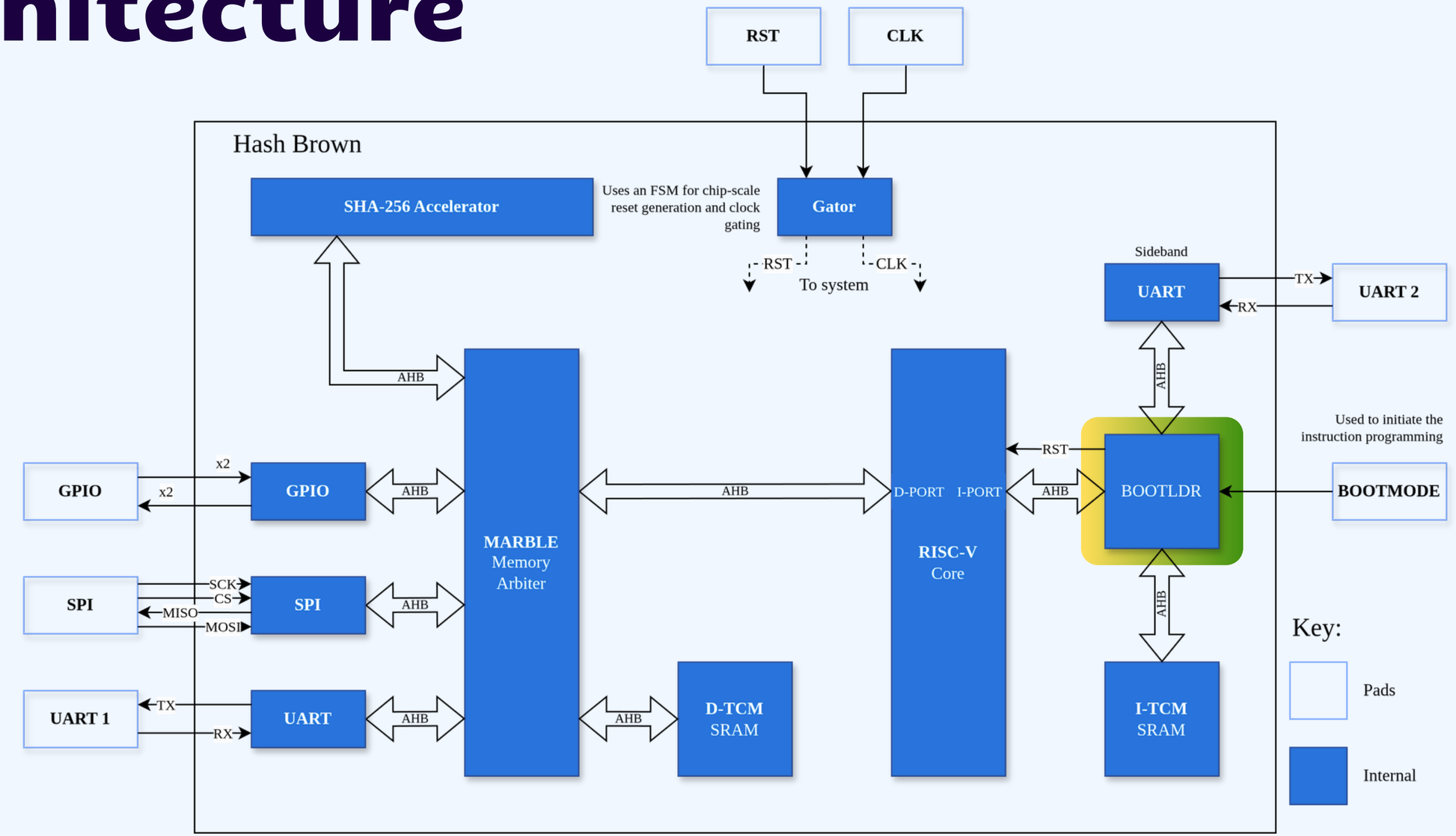
- 0x00 reset | 0x04 data byte | 0x08 last flag | 0x0C done
- 0x10-0x2C eight 32-bit hash words
- Zero-wait-state
  - CPU drives it with plain load/store

End-to-end use cases (on the SoC)

- Integrity/checksums
- Secure boot
- HMAC
- Blockchain



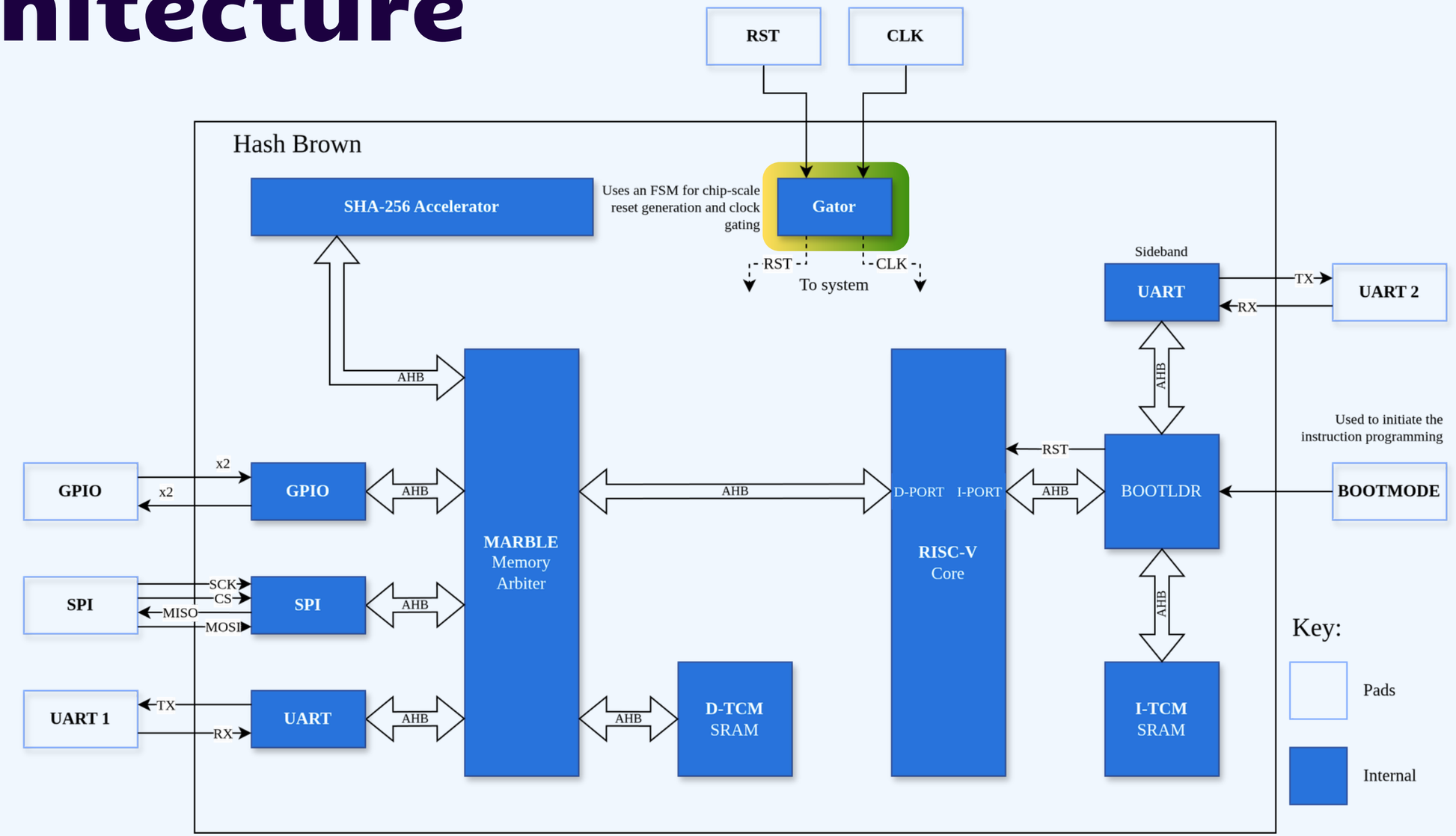
Architecture

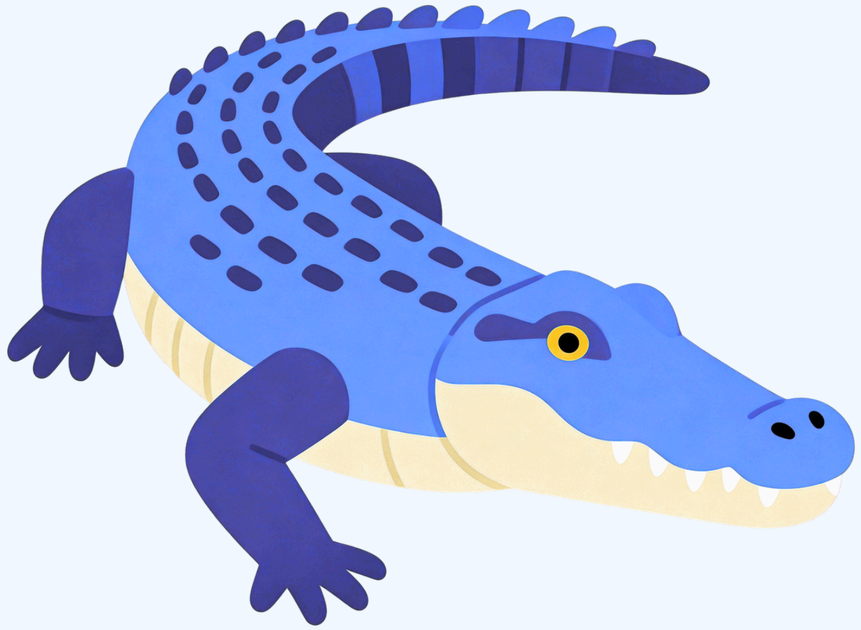


Bootloader

- Uses a secondary hardened UART
- Fixed baud rate & configuration
- Isolated, no RISC-V access
- Echos back instructions for verification
- Takes control of the I-TCM to inject words

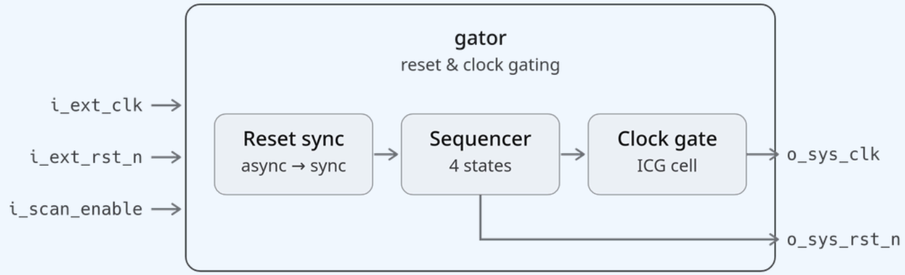
Architecture



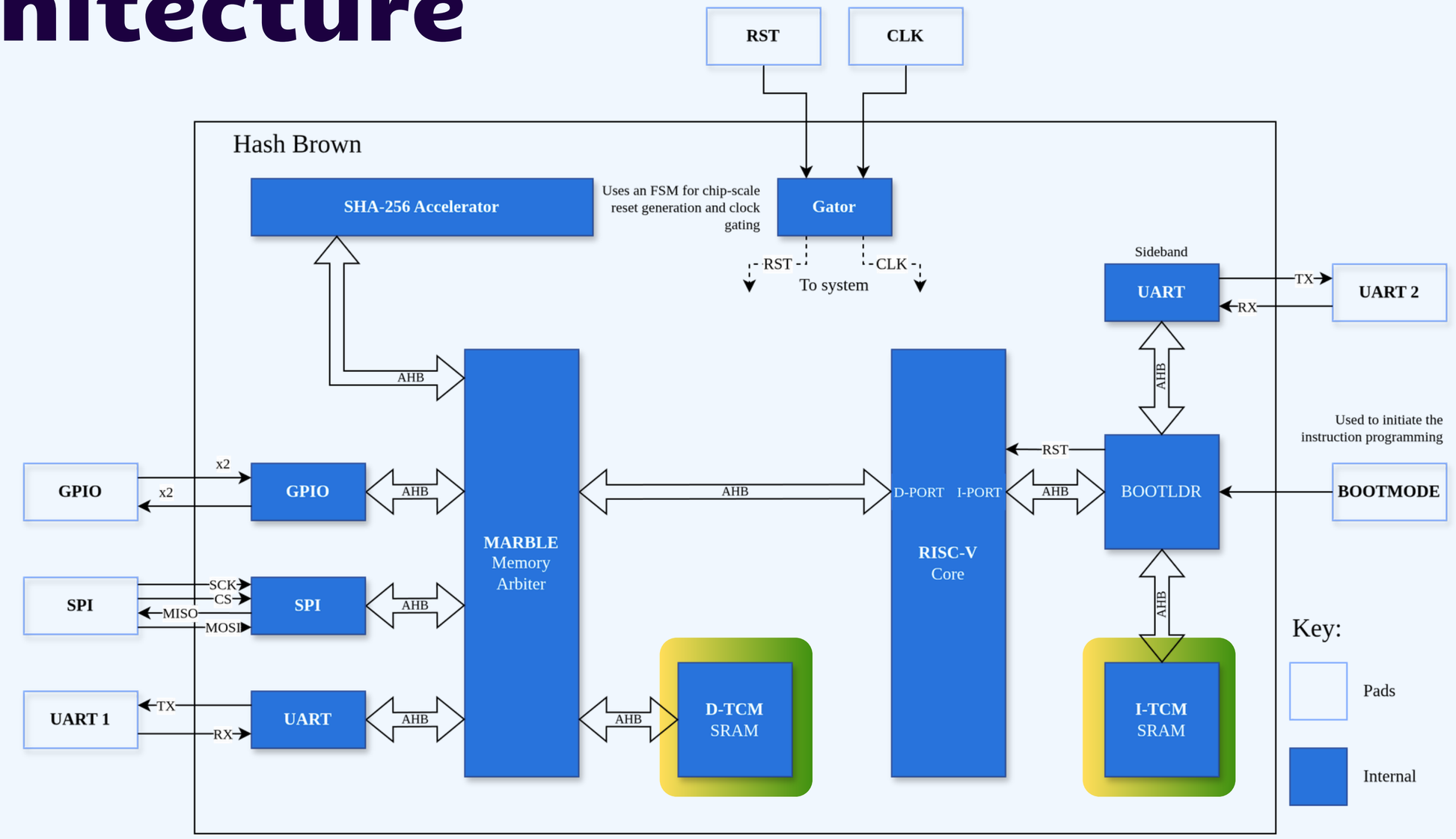


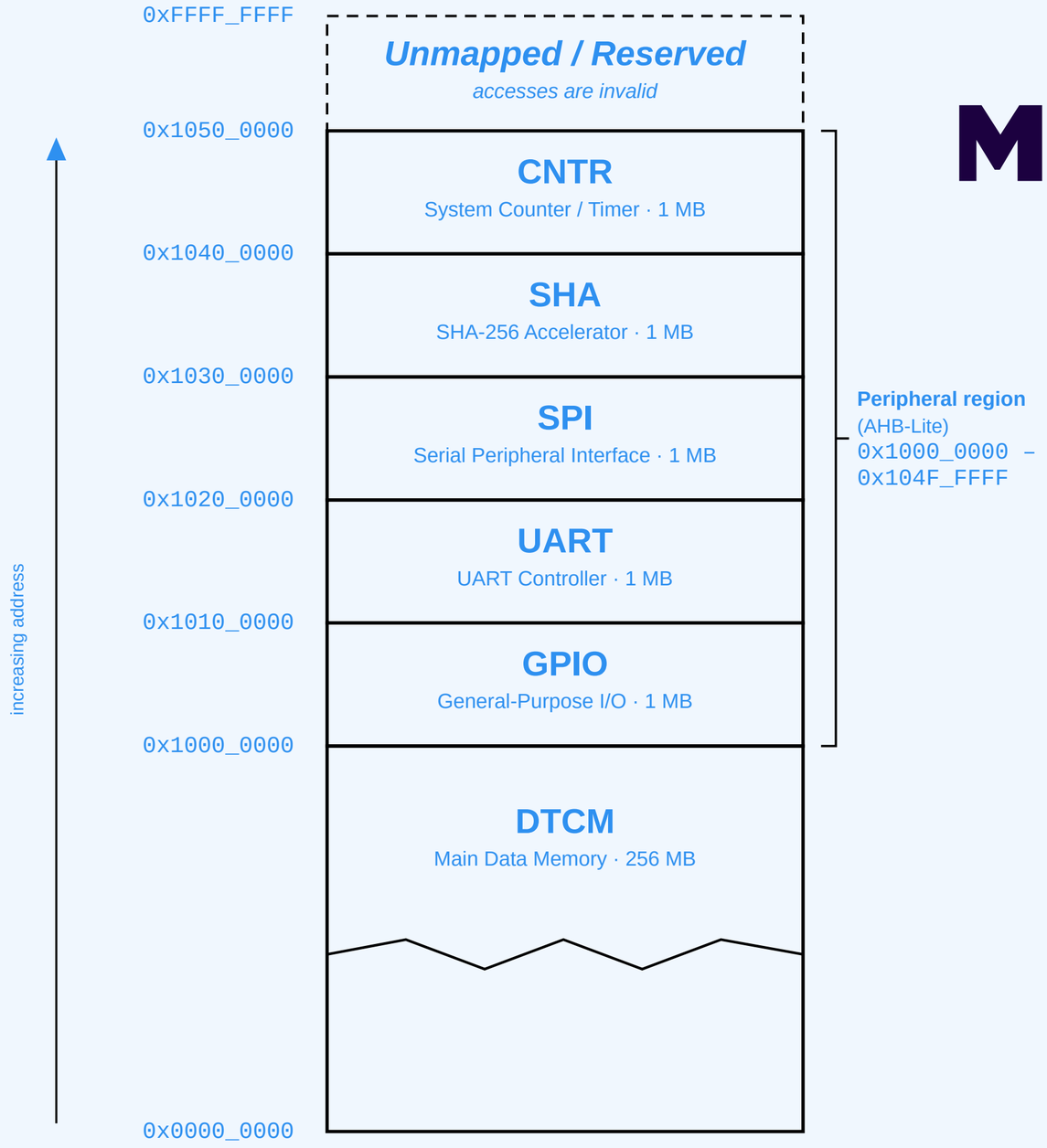
Gator

- Chip wide clock gating.
- FSM to meet reset recovery timing constraints.
- Chip wide async reset provider.



Architecture





Memory & I/O

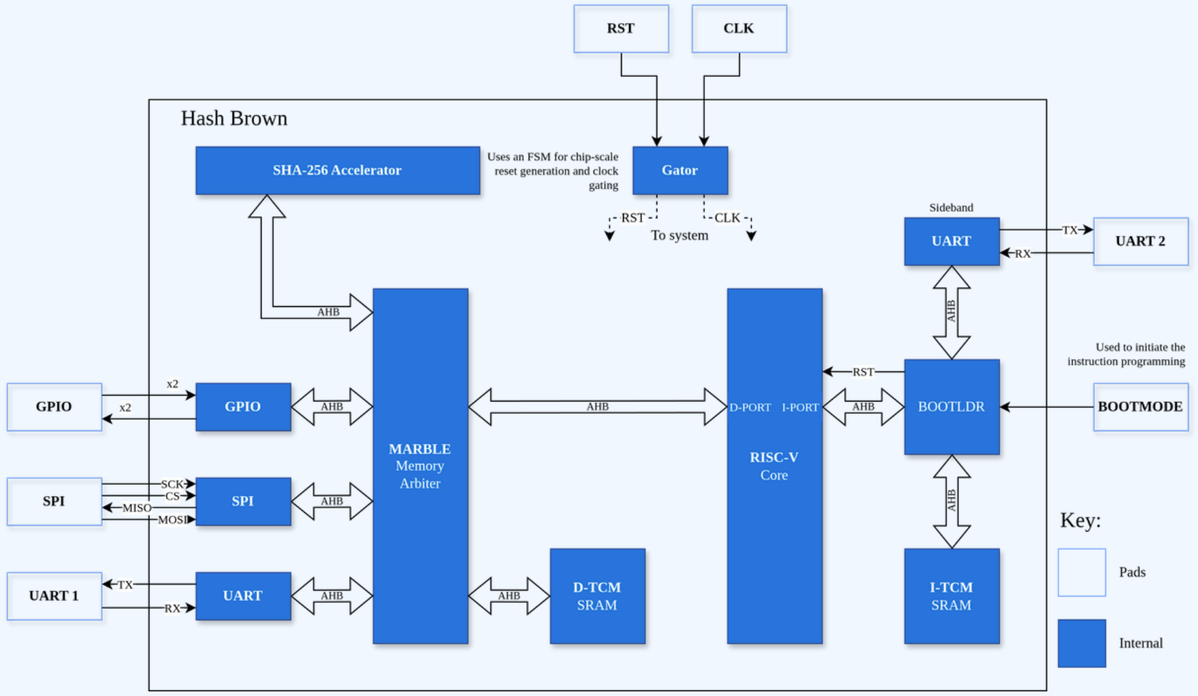
- 16 KB I-TCM – Compiled
- 1 KB D-TCM – Register File
- MMIO is used to interact with all blocks

Communication and Interfacing

01 Internal and External Communication

02 AHB-Lite Internal Communication Protocol

03 Use of GPIO, SPI and UART for External Communications



Testing and Verification

Pre-Silicon Verification

01 Functional and Code Coverage → Cocotb

02 Formal Verification

03 Regressions

04 Early Synthesis Tests → YoSys

05
RISCV → Fully Verified
SHA-256 → Partially Verified
UART → Fully Verified
SPI → Partially Verified

Pre-Silicon Testing

01 Design Rule Checking

02 Layout vs. Schematic

03 FPGA Synthesis

04 Gate-Level Sim

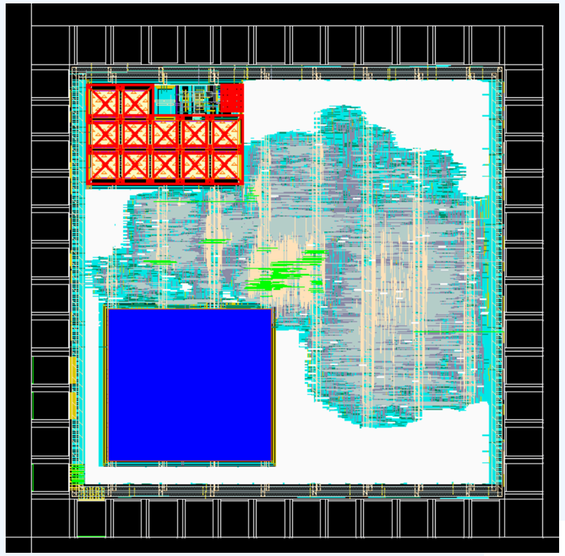
Post-Silicon Testing

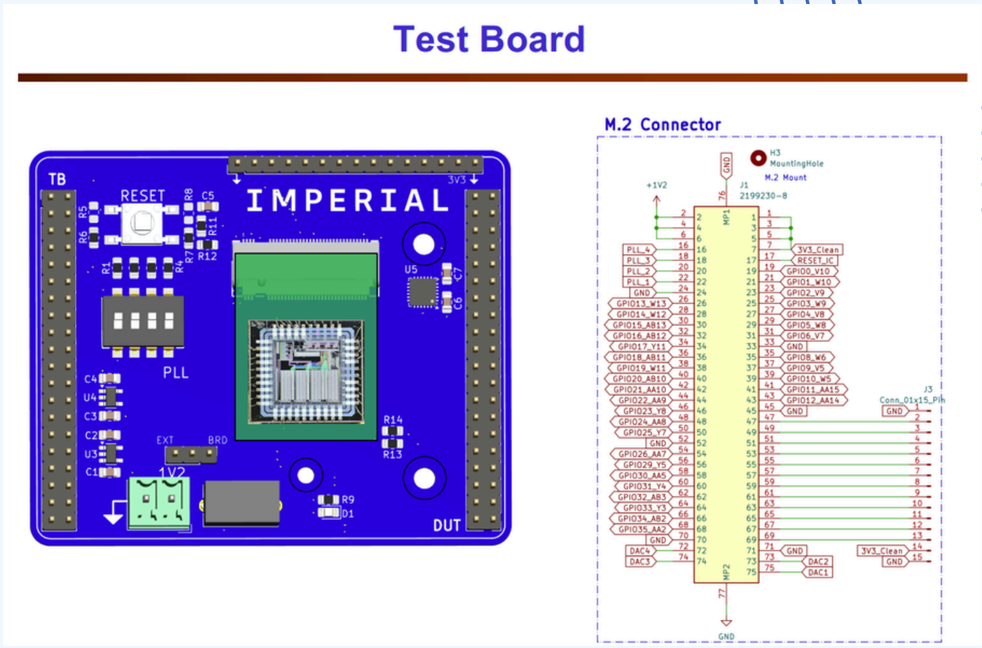
01 BIST

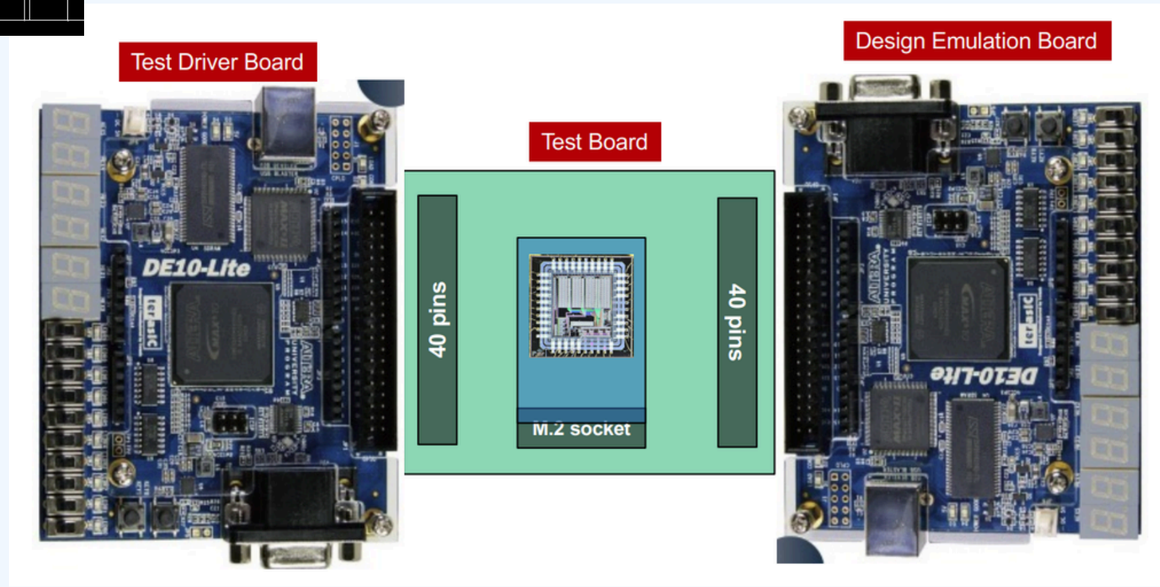
02 Scan Chains/ATPG

03 Test Dock/FPGA Interface

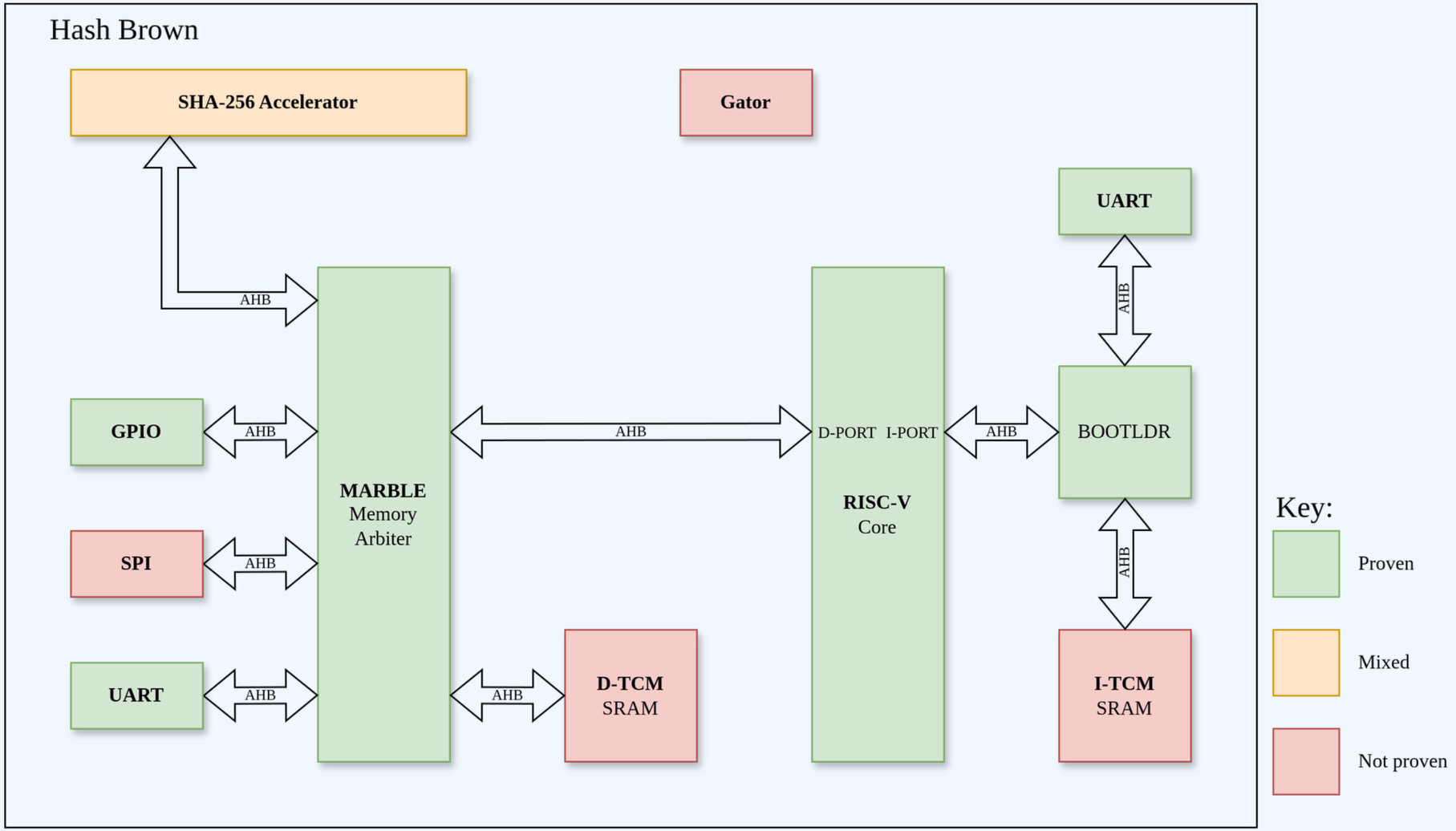
04 Power/IR drop, Probe Nodes, Functional testing etc





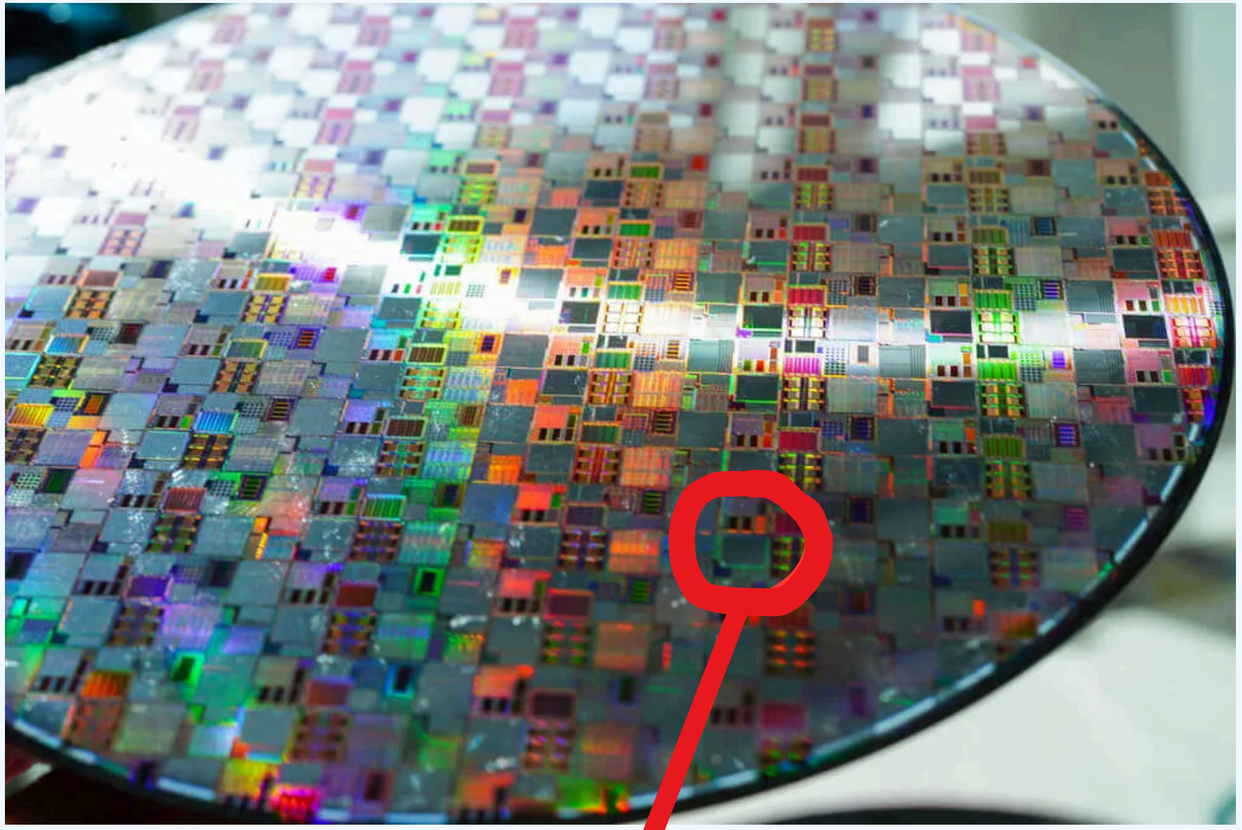


Formal Coverage



Summary

- Better time management
- Hands-on experience with industry-standard EDA tools
- Real-world exposure
- Creative freedom
- Theory into silicon
- Verification matters
- Iteration for synthesis and timing closure
- Teamwork



Not our chip

Thank You!

Thank you to Apple & Prof. Cheung